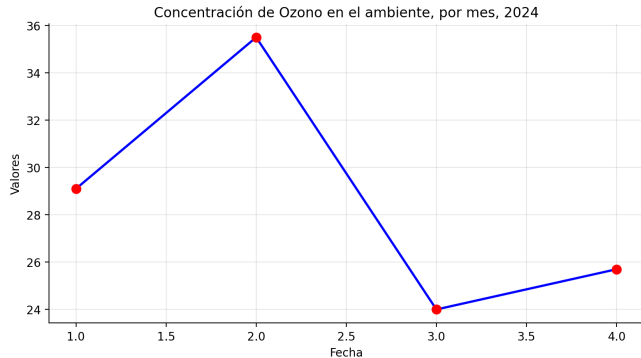


Las partes de un gráfico

Introducción a la Ciencia de Datos — versión R / ggplot2

Primera parte: un gráfico, una serie

El gráfico del que partimos



Concentración media mensual de ozono, de enero a abril. Una sola serie, cuatro observaciones.

El código que lo produce

```
ozono_mes %>%  
  ggplot(aes(x = mes, y = o3_medio_mes)) +      # datos y mapeo  
  geom_line(color = "blue", linewidth = 1) +    # línea  
  geom_point(color = "red", size = 3) +          # puntos  
  theme_minimal() +                             # diseño limpio  
  labs(  
    title = "Concentración de Ozono en el ambiente, por mes, 2024",  
    x = "Fecha",  
    y = "Valores"  
  )
```

Cada línea de este código controla una parte distinta del gráfico. Las vamos a recorrer una por una.

Un objeto que se construye por capas

```
p <- ggplot(ozono_mes, aes(x = mes, y = o3_medio_mes)) # todavía vacío
p + geom_line() # una capa
p + geom_line() + geom_point() # dos capas
```

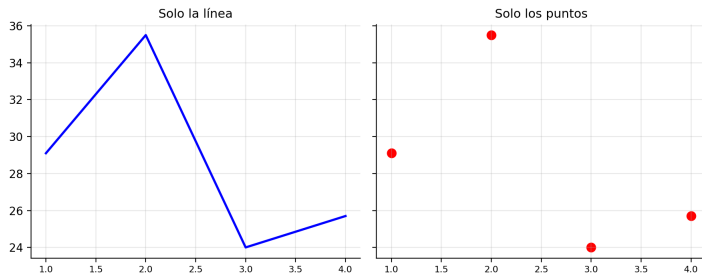
- ▶ `ggplot()` devuelve un objeto: los datos y el mapeo, sin nada dibujado.
- ▶ Cada `+` agrega una capa o una modificación. El orden importa: lo último se dibuja encima.
- ▶ El objeto se puede guardar en una variable, seguir sumándole cosas y mostrarlo cuando se quiera. No hay `fig` y `ax` por separado: el gráfico es una sola cosa que crece.

Los datos y el mapeo

```
ggplot(ozono_mes, aes(x = mes, y = o3_medio_mes))
```

- ▶ Los datos son la única parte que aporta información nueva; todo lo demás sirve para leerla.
- ▶ `aes()` es el **mapeo**: conecta una columna de la tabla con una propiedad visual. Acá, `mes` con la posición horizontal y `o3_medio_mes` con la vertical.
- ▶ Lo que va dentro de `aes()` se lee de los datos, fila por fila. Cuatro pares (`mes`, `concentración`) traducidos a posiciones dentro del área de los ejes.

La geometría: dos capas superpuestas



- ▶ `geom_line()` afirma que hay continuidad entre meses consecutivos y dirige la lectura hacia la tendencia.
- ▶ `geom_point()` marca dónde hay una medición efectiva y frena la lectura de valores intermedios que nadie observó.
- ▶ Juntos: se ve la tendencia sin perder de vista que los datos son discretos. Las dos capas comparten el mapeo que se declaró en `ggplot()`.

Atributos estéticos: fuera o dentro de aes()

```
geom_line(color = "blue", linewidth = 1)      # fijo: fuera de aes()  
geom_point(color = "red", size = 3)           # fijo: fuera de aes()
```

- ▶ color, linewidth y size puestos así no codifican ninguna variable: son decisiones de legibilidad, no información.
- ▶ El mismo atributo pasa a ser informativo cuando va **dentro** de aes() y recibe una columna: aes(color = estacion). Entonces ggplot asigna un color por valor y agrega la leyenda.
- ▶ Esa distinción (fuera de aes(): fijo; dentro: mapeado) es la regla más importante de ggplot.

El título y las etiquetas

```
labs(title = "Concentración de Ozono en el ambiente, por mes, 2024",  
      x = "Fecha", y = "Valores")
```

- ▶ El título dice **qué** se está midiendo, **de quién** y **cuándo**. También puede afirmar el hallazgo (“El ozono alcanza su máximo en verano”); eso exige estar seguro de lo que el gráfico muestra.
- ▶ Las etiquetas de eje son el punto débil del gráfico. “Valores” no informa nada: falta la variable y falta la unidad.
- ▶ El eje vertical debería decir qué se mide y en qué unidad; el horizontal, que la unidad de tiempo es el mes.
- ▶ `labs()` también controla el nombre de la leyenda (`color = "Estación"`) y el pie (`caption = "Fuente: ..."`).

Las marcas y sus etiquetas

- ▶ El eje horizontal muestra marcas en 1.5, 2.5 y 3.5: meses que no existen. Pasa porque mes es numérico y ggplot lo trata como continuo.
- ▶ Se corrige con la escala del eje:

```
scale_x_continuous(breaks = 1:4, labels = c("Ene", "Feb", "Mar", "Abr"))  
scale_y_continuous(limits = c(0, 40))
```

- ▶ breaks fija dónde van las marcas; labels, qué dicen. limits fija el rango.
- ▶ El eje vertical tampoco arranca en cero: admisible, pero exagera la variación. Con `limits = c(0, 40)` la caída de febrero a marzo se ve del tamaño que tiene.
- ▶ Las escalas (`scale_x_*`, `scale_y_*`, `scale_color_*`) son donde se decide cómo un valor se traduce a una posición o a un color.

La grilla y los bordes

```
theme_minimal()  
theme(panel.grid.minor = element_blank())
```

- ▶ El tema controla todo lo que no son datos: fondo, grilla, bordes, tipografía.
- ▶ `theme_minimal()` ya hace lo que uno quiere por defecto: grilla tenue, sin marco, sin fondo gris. Es la razón por la que el código de partida es corto.
- ▶ La grilla ayuda a estimar valores concretos sin recorrer el eje con la vista. La secundaria (`panel.grid.minor`) suele sobrar: marca posiciones donde no hay valores.
- ▶ Criterio general: quitar todo elemento que no ayude a leer un valor. `theme()` permite ajustar uno por uno; se usa después de `theme_minimal()`, no en lugar de.

Lo que este gráfico todavía no tiene

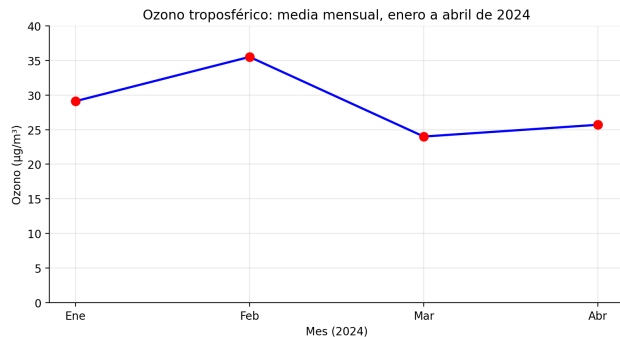
- ▶ **Unidades:** sin ellas no se puede juzgar si los valores son altos o bajos.
- ▶ **Fuente de los datos:** de dónde salen las mediciones y quién las produjo. En ggplot va en `labs(caption = ...)`.
- ▶ **Leyenda:** acá no hace falta, porque hay una sola serie. Con dos o más pasa a ser obligatoria, y ggplot la genera sola en cuanto algo va dentro de `aes()`.
- ▶ **Nota al pie:** aclaraciones sobre cobertura, faltantes o cambios de método.

La figura es un objeto

```
p <- ozono_mes %>%  
  ggplot(aes(x = mes, y = o3_medio_mes)) +  
  geom_line() + geom_point()  
p                                     # se dibuja al imprimirlo  
ggsave("ozono_2024.png", p, width = 8, height = 5,  
        dpi = 300, bg = "white")
```

- ▶ Un gráfico asignado a una variable no se dibuja; se dibuja al escribir su nombre o con `print(p)`.
- ▶ `ggsave()` guarda el objeto: no depende de lo que haya en pantalla. `width` y `height` en pulgadas; `dpi = 300` es lo mínimo razonable para imprimir o proyectar.
- ▶ `bg = "white"`: `theme_minimal()` deja el fondo transparente, y en algunos visores se ve negro.
- ▶ Para rehacerla con un cambio: `p + scale_y_continuous(limits = c(0, 40))` y guardar de nuevo. El original no se toca.

Una versión corregida



Fuente: valores leídos del gráfico original.

Mismos datos, mismas geometrías. Cambian el título, las etiquetas, las marcas del eje horizontal, el rango del eje vertical y la fuente al pie.

Resumen de la primera parte

Parte	Qué controla	Dónde se toca
Datos y mapeo	qué columna va a qué propiedad	<code>ggplot(datos, aes(...))</code>
Geometría	cómo se dibujan	<code>geom_line()</code> , <code>geom_point()</code>
Atributos fijos	color, grosor, tamaño	<code>color=</code> , <code>size=</code> fuera de <code>aes()</code>
Título y etiquetas	qué, de quién, cuándo; unidad	<code>labs()</code>
Marcas y rango	posición a valor	<code>scale_x_*(breaks, labels, limits)</code>
Grilla y bordes	todo lo que no son datos	<code>theme_minimal()</code> , <code>theme()</code>
Pie	fuentes y notas	<code>labs(caption = ...)</code>
Guardado	archivo de salida	<code>ggsave()</code>

Segunda parte: más de una serie

Qué cambia cuando hay más de una serie

Con una serie, todas las decisiones eran de legibilidad. Con dos o más, aparecen decisiones de **codificación**: qué variable va al color, qué va a un panel aparte, en qué escala se lee cada eje.

- ▶ Un color por grupo: la variable entra en `aes()`.
- ▶ Superponer o apilar: mismo eje o paneles.
- ▶ Tres geometrías y la pregunta que responde cada una.
- ▶ Escalas: logarítmica y marcas a mano.
- ▶ Color continuo y color discreto.
- ▶ Paneles con `facet_wrap()`: la misma pregunta, varias veces.

Los ejemplos son los de la clase 9: ozono y PM2.5 en Montevideo, enero a abril de 2024, Catálogo de Datos Abiertos.

Un color por grupo

```
ggplot(o3_h, aes(x = fecha_hora, y = media, color = estacion)) +  
  geom_line(linewidth = 0.3) +  
  labs(color = "Estación")
```

- ▶ `color = estacion` **dentro** de `aes()`: ggplot parte los datos por estación, dibuja una línea por grupo, asigna un color a cada una y agrega la leyenda. Una línea de código.
- ▶ No hay bucle: en ggplot se declara qué variable distingue los grupos y el resto lo hace la gramática.
- ▶ `labs(color = ...)` pone el título de la leyenda. Con dos o más series, la leyenda deja de ser opcional.

Lo mismo vale para `fill` (barras, cajas), `shape` (puntos) y `linetype`.

Superponer o apilar

Dos series se pueden poner **en el mismo panel** o **en paneles distintos**.

- ▶ **Mismo panel:** se comparan niveles. Solo si tienen la misma unidad y rangos parecidos. Ozono por estación: sí, `aes(color = estacion)`.
- ▶ **Paneles apilados:** se comparan formas. Cuando la unidad o el rango difieren. Ozono y PM2.5: sí, apilados.

```
p_o3 / p_pm                                # patchwork: "/" apila, "|"
facet_wrap(~ contaminante, ncol = 1, scales = "free_y")
```

- ▶ Dos caminos: `patchwork` combina dos gráficos ya hechos; `facet_wrap()` parte un solo gráfico por una variable. Para facetar, los dos contaminantes tienen que estar en la misma tabla, con una columna que diga cuál es cuál (`bind_rows`).
- ▶ `scales = "free_y"`: cada panel con su propio eje vertical. Sin eso, `ggplot` comparte escalas, y ozono (0-120) aplasta a PM2.5 (3-50).

Varios paneles con facet_wrap()

```
ggplot(aire, aes(x = pm2_5, y = o3)) +  
  geom_point(size = 0.8, alpha = 0.4) +  
  facet_wrap(~ panel)
```

- ▶ Un panel por valor de la variable. Título, ejes y grilla se repiten solos; las etiquetas de eje van una sola vez.
- ▶ Por defecto los paneles **comparten escalas**: es lo correcto cuando se van a comparar. `scales = "free"` es la excepción, no la regla; con escalas distintas la vista compara formas que no son comparables.
- ▶ Para poner un número por panel (el r, el n), se calcula en una tabla aparte y se arma la etiqueta del panel con `sprintf`: el título del panel es una columna más.

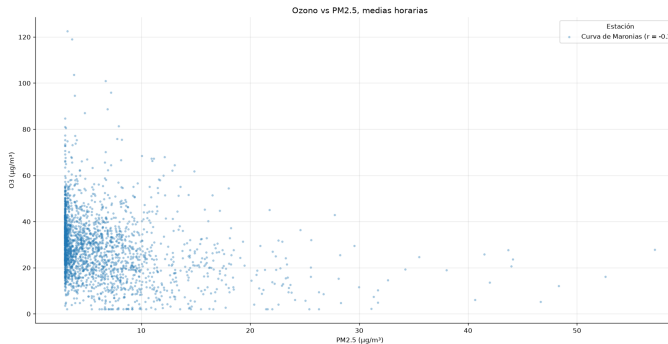
Tres geometrías, tres preguntas

Geometría	Función	Pregunta que responde
Línea	<code>geom_line()</code>	¿Cómo cambia a lo largo de un eje ordenado?
Puntos	<code>geom_point()</code>	¿Cómo se relacionan dos variables?
Cajas	<code>geom_boxplot()</code>	¿Cómo se distribuye, por grupo?

- ▶ La línea afirma continuidad entre observaciones consecutivas. Solo tiene sentido si el eje horizontal es ordenado: tiempo, o una variable continua.
- ▶ Los puntos no afirman nada entre observaciones: cada uno es una medición.
- ▶ La caja resume: mediana, cuartiles, extremos. Pierde el detalle, gana comparación. Pide una variable discreta en x (`month(día, label = TRUE)`)).

`geom_col()` es una línea sin continuidad: categorías, no un eje ordenado.

Dispersión: dos variables



- ▶ Cada punto es una hora. `size = 0.8` y `alpha = 0.3` para que se vea dónde hay muchos.
- ▶ La pared vertical en $\text{PM}_{2.5} = 3$ no es atmósfera: es el piso del sensor.
- ▶ El `r` en el título viene de `cor()`, calculado antes y pegado con `sprintf`. El título no se escribe a mano si el número puede cambiar.

Densidad: cuando hay miles de puntos

Con 2.800 puntos, `geom_point()` sin cuidado es una mancha.

- ▶ `alpha = 0.3`: transparencia; donde se apilan, se oscurece.
- ▶ `size = 0.8`: chico. El tamaño por defecto (1.5) es para decenas de puntos, no miles.
- ▶ `coord_cartesian(xlim = ..., ylim = ...)`: recortar la zona vacía **sin descartar datos**. `scale_*_continuous(limits = ...)` los descarta, y las medias o cajas calculadas después cambian.
- ▶ Si aun así no se ve, `geom_hex()` cuenta puntos por celda y colorea el conteo. Es un histograma en dos dimensiones.

El objetivo es que el lector vea **dónde está la masa** y **dónde están los pocos**. Un gráfico donde todo es igual de negro no muestra ninguna de las dos.

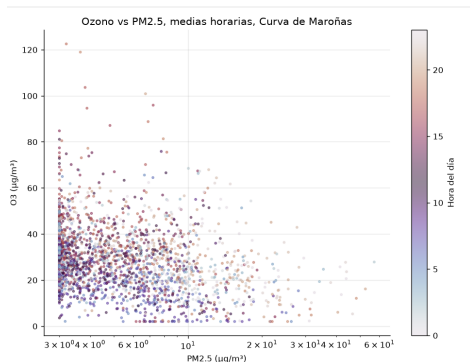
Escalas: cuándo usar logaritmo

PM2.5 va de 3 a 55, con casi todo entre 3 y 15. En escala lineal, el 80 % de los puntos ocupa el 20 % del ancho.

```
scale_x_log10(breaks = c(3, 5, 10, 20, 30, 50))
```

- ▶ La escala logarítmica **separa lo chico y comprime lo grande**. Sirve para variables asimétricas a la derecha: concentraciones, ingresos, tamaños.
- ▶ breaks a mano en valores redondos: sin eso, ggplot marca 3, 10, 30 y el lector no tiene el 5 ni el 20 para apoyarse.
- ▶ La escala se cambia en el eje, no en los datos: `log(pm2_5)` en `aes()` da el mismo dibujo con marcas ilegibles (1.1, 2.3, 3.4).
- ▶ Se usa en un eje si esa variable lo pide; no por costumbre.

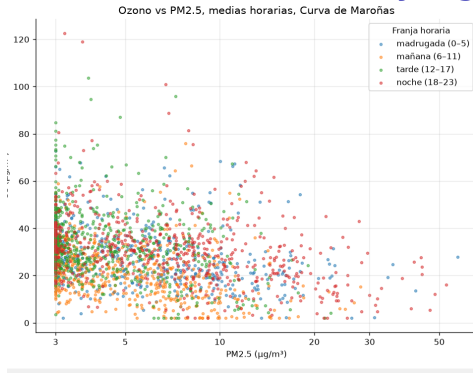
Color continuo: una variable numérica



```
ggplot(aire,  
  aes(x = pm2_5, y = o3,  
      color = hour(fecha_hora))) +  
  geom_point(size = 0.8, alpha = 0.5) +  
  scale_color_viridis_c() +  
  labs(color = "Hora del día")
```

- ▶ Una columna numérica en `aes(color = ...)`: ggplot elige una escala continua y dibuja la barra de color como leyenda.
- ▶ `scale_color_viridis_c()`: paleta legible y ordenada. La hora es cíclica (0 y 23 se parecen) y viridis no lo es; el borde entre 23 y 0 es un salto de color que no está en los datos.

Color discreto: cortar y agrupar

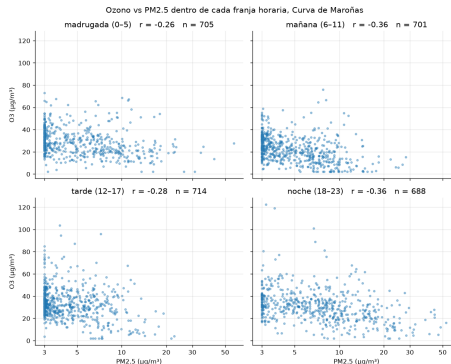


```
aire <- aire %>%  
  mutate(franja = cut(  
    hour(fecha_hora),  
    breaks = c(-1, 5, 11, 17, 23),  
    labels = c("madrugada", "mañana",  
               "tarde", "noche")))
```

```
ggplot(aire,  
  aes(x = pm2_5, y = o3,  
       color = franja)) +  
  geom_point(size = 0.8, alpha = 0.5)
```

- ▶ `cut()` convierte la hora en un factor de cuatro niveles; ggplot pasa a escala discreta y leyenda con cuatro entradas.
- ▶ **Los cortes son una decisión.** En enero el sol se pone después de las 20: la franja “noche” contiene tardes. Cambiar `breaks` cambia lo que se ve.

Paneles: la misma pregunta, varias veces



```
r_franja <- aire %>%  
  group_by(franja) %>%  
  summarise(r = cor(o3, pm2_5)) %>%  
  mutate(panel = sprintf(  
    "%s r = %.2f", franja, r))
```

```
aire %>%  
  left_join(r_franja, by = "franja") %>%  
  ggplot(aes(x = pm2_5, y = o3)) +  
  geom_point(size = 0.8, alpha = 0.4) +  
  facet_wrap(~ panel)
```

- Un panel por franja, mismos ejes. Dentro de cada uno, r entre $-0,26$ y $-0,36$; sin partir, $-0,20$. El r en la etiqueta del panel convierte cuatro imágenes en cuatro números comparables.

Tres figuras, tres lecturas del mismo dato

Figura	Lo que muestra	Lo que afirma
Dispersión simple Color por franja	una nube con $r = -0,2$ los extremos son de horas distintas	hay una asociación débil la hora está mezclada
Paneles por franja	$r \approx -0,3$ en cada uno	la hora diluía la asociación

- ▶ Los datos son los mismos. Cambió qué variable entró en `aes(color = ...)` y si se facetó o no.
- ▶ Una figura **sostiene una afirmación** cuando la afirmación se lee sin que haya que explicarla. Las tres son correctas; solo la tercera sostiene la última fila.
- ▶ El título puede decir la afirmación (“Dentro de cada franja, más PM2.5 va con menos ozono”). Exige estar seguro.

Antes de entregar

- ▶ **Título:** qué se mide, de quién, cuándo. O la afirmación, si está segura.
- ▶ **Unidades** en cada eje. Sin ellas no se sabe si un valor es alto o bajo.
- ▶ **Leyenda** con nombre (`labs(color = ...)`); la que ggplot genera sola lleva el nombre de la columna, que rara vez es el que el lector necesita.
- ▶ **Marcas** en valores que existen, con etiquetas que el lector use.
- ▶ **Fuente** al pie: `labs(caption = ...)`.
- ▶ **Nota** sobre cobertura, faltantes o decisiones: qué estación, qué período, cómo se agregó, con qué cortes.

Y una pregunta antes de guardar: ¿qué afirmación sostiene esta figura, y se lee sin que yo esté al lado para explicarla?

Resumen de la segunda parte

Parte	Qué controla	Dónde se toca
Grupos	un color por categoría	<code>aes(color = variable)</code>
Apilar Paneles	dos gráficos en uno la misma pregunta varias veces	<code>patchwork: p1 / p2</code> <code>facet_wrap(~ variable)</code>
Dispersión densa	la masa y los pocos	<code>size=, alpha=,</code> <code>geom_hex()</code>
Escala	separar lo chico	<code>scale_x_log10(breaks = ...)</code>
Color continuo Color discreto	variable numérica variable cortada	<code>scale_color_viridis_c()</code> <code>cut() + aes(color = factor)</code>
Recorte	ver una zona sin descartar	<code>coord_cartesian()</code>

Figuras de la primera parte: valores leídos del gráfico original (enero a abril de 2024). Figuras de la segunda parte: ozono y PM2.5, Curva de Maroñas, enero a abril de 2024, Catálogo de Datos 